ALIS: how SMART-G computes many wavelengths from one set of photons

Absorption Lines Importance Sampling (Emde et al.). A reference for the SatelliteArray project: what the method does, how SMART-G exposes it, what it can and cannot do for our renders, and what has and has not been tested here.

1. The problem it solves

A Monte Carlo radiative-transfer code follows photons through the atmosphere. The path of a photon is decided by **scattering** (by air molecules, aerosols and cloud droplets), and light is lost to **absorption** by gases such as O_2, H_2O and O_3. Scattering varies smoothly with wavelength. Gas absorption does not: across a narrow band it jumps between nearly nothing and almost total absorption, over hundreds of narrow lines (Figure 2).

The straightforward approach simulates every wavelength separately, so a band with M absorption features costs M simulations. ALIS exploits the fact that the scattering is almost the same at all wavelengths of a narrow band, so the *same* photon paths can be reused for all of them.

2. The idea

Draw each photon path once, then give it a separate weight for every wavelength. The path (where the photon scatters and in which direction) is sampled with the scattering properties of the band. The absorption that differs from wavelength to wavelength is not used to decide the path. Instead, the photon is given, for each wavelength, a weight equal to the fraction of its light that survives the absorption of that wavelength along the very same path (a Beer-Lambert factor). One photon therefore contributes to the result at every wavelength.

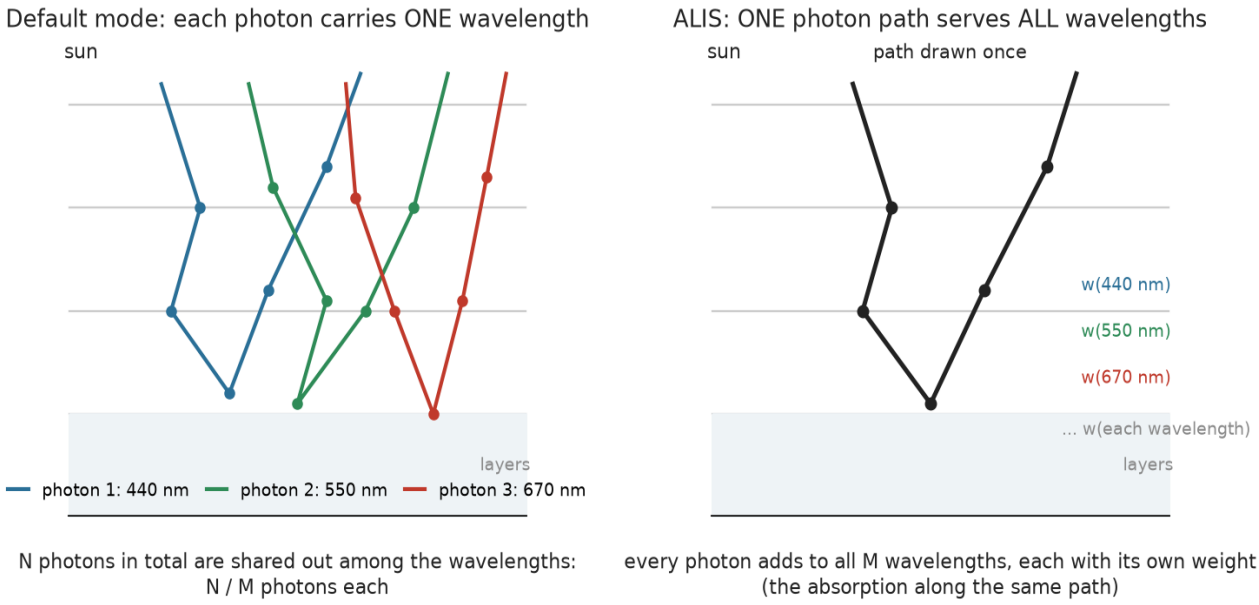


Figure 1. Schematic. Left: the default mode, in which each photon is assigned one wavelength. Right: ALIS, in which a single path carries a weight for every wavelength. (Illustration of the principle, not output of SMART-G.)

3. How it works, step by step

- **Spectral grids.** Two wavelength grids are built (`smartg.bandset.spectral_grids`): a high-resolution one, on which the gas absorption is evaluated, and a low-resolution one, on which the scattering is evaluated. The scattering at each high-resolution wavelength is obtained by linear interpolation between the low-resolution ones.
- **Launching.** A photon is launched with a randomly chosen starting wavelength (uniform by default). In ALIS the launch counts for *all* wavelengths (the kernel increments the photon counter of every wavelength), because the photon represents all of them.
- **Propagation.** The photon is followed layer by layer (the plane-parallel scheme, option `alt_pp`), and scatters according to the scattering properties. Along the way, for each wavelength, the accumulated absorption optical depth is tracked.

- **Weights.** When the photon (or a local estimate towards the sun or the sensor) contributes to the result, the contribution at wavelength λ_i is multiplied by that wavelength's weight, roughly $\exp(-\sum k_{abs}(\lambda_i)\cdot s)$ over the path, together with the scattering correction from the interpolation. The result is one value per wavelength.
- **Result.** A full high-resolution spectrum from one set of photons. The noise at neighbouring wavelengths is correlated, because they share the paths.

The exact weight formulas in SMART-G were not read in detail; this describes the method as published and as the source code is organised.

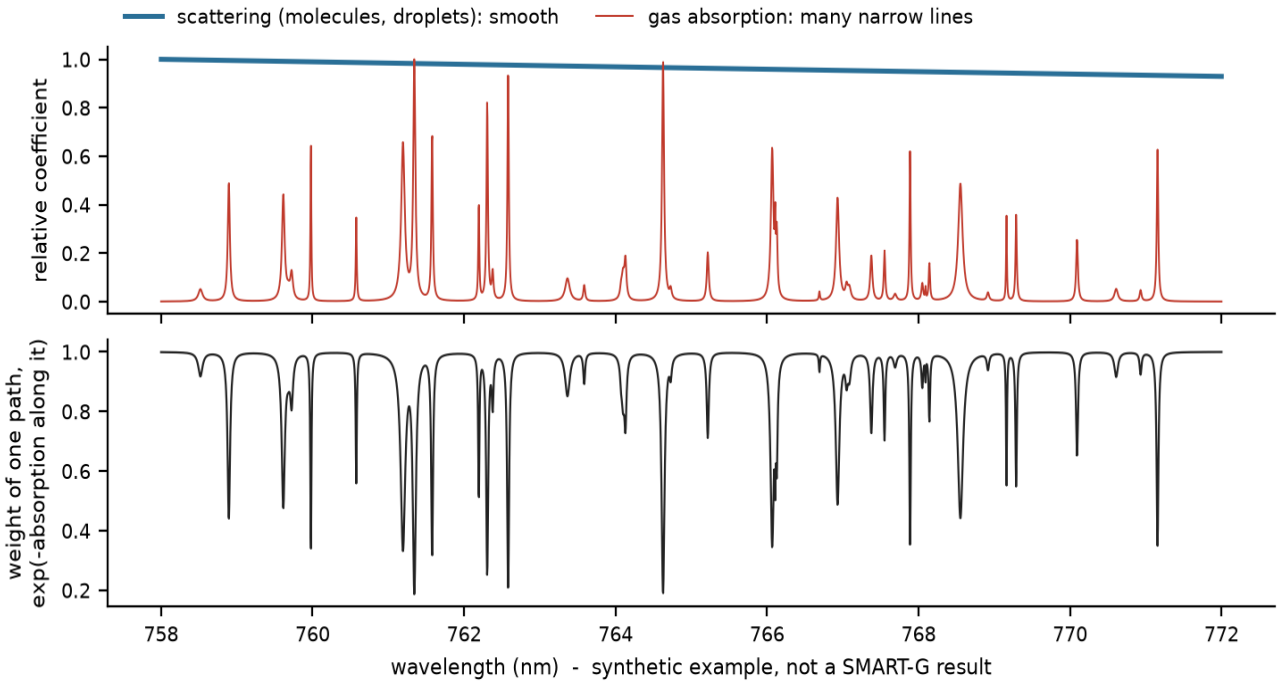


Figure 2. Why it works. The scattering that decides a path is smooth across the band (top, blue), so one path is valid at all wavelengths. The absorption (top, red) is what changes, and it enters only as the per-wavelength weight (bottom). Synthetic data, for illustration only.

4. Default mode compared with ALIS

	Default mode of SMART-G	ALIS
Wavelength of a photon	One, drawn at random from the list (uniform, or by a distribution you give with `wavelength_proba`).	All of them: a launched photon counts for every wavelength.
Photons per wavelength	N / M for N photons and M wavelengths.	N for each wavelength (the same photons).
Noise at one wavelength	Grows as √M at fixed N.	About that of N photons; correlated between wavelengths.
Cost of extra wavelengths	Almost none in time at fixed N, but each wavelength gets fewer photons.	A weight is computed per wavelength along each path; the plane-parallel scheme it needs is slower.
Needs	Nothing special.	SMART-G compiled with `alis=True` and `alt_pp=True`.
Assumption	None about the spectrum.	Scattering (nearly) the same across the band.

Measured here. In the default mode, the green band of a three-wavelength render of the cropped BOMEX cloud had 1.90 times the Monte Carlo noise of a single-wavelength render at the same photons per pixel (two runs with different seeds, noise taken from their difference). Splitting the photons predicts √3 = 1.73. Run time was about the same (roughly 58 s against 47 s), because the total number of photons is what costs time.

5. ALIS in SMART-G

Enable it when the engine is created, and pass the options to run:

```
from smartg.smartg import Smartg, Alis
sg = Smartg(back=True, opt3d=True, alis=True, alt_pp=True) # compile-time options
ds = sg.run(wavelength=wl, atmosphere=prof, surface=surf, sensor=sensors, le=le,
n_photons=n, alis_options=Alis(n_low=2))
```

Option (class `Alis`)	**Meaning**
n_low	Number of low-spectral-resolution computations of the scattering, at least 2; or −1 to compute every wavelength. The scattering at the other wavelengths is interpolated.
hist, max_hist	Record the photon histories (up to max_hist, default 8×10^6) so that `smartg.histories` can rebuild high-resolution Stokes vectors and derive air-mass-factor statistics afterwards.
n_jac, n_jac_abs	Also compute Jacobians: the same photons are weighted for perturbed atmospheric profiles. With n_jac_abs the perturbation is for absorption only, which allows a small n_low.
(engine) amf_variance, cdist_wabs, nscl	Further outputs for air-mass factors; they require alis=True.

6. When it helps, and what it does not do

- **Helps** for a narrow band with many absorption lines (the O_2 A-band near 760 nm, water-vapour bands), where hundreds of wavelengths are needed and the scattering is nearly constant.
- **Less clear-cut for broad visible bands.** Blue, green and red differ in molecular scattering, which is what ALIS assumes constant, and a visible satellite channel is described by a single representative wavelength (see REPTRAN below), so there is nothing to share *within* a channel. One can still run all the channels' wavelengths in one ALIS run, so that the photons are shared across the channels. In one test on this project's scene (section 7) that lowered the noise at the same run time, but its accuracy is not yet established.
- **It does not apply the sun or a sensor's spectral response.** The output is the reflectance at each wavelength. The Sun's spectrum and a camera response are combined afterwards, as weights on the per-wavelength results (for example `reduce_reptran` with `use_solar` and `extern_weights`). Because the estimate is linear in these weights, applying them afterwards is equivalent to putting them on the photons.
- **It is not REPTRAN or KDIS.** REPTRAN and KDIS reduce the *number of wavelengths* a band needs (a channel becomes a few representative wavelengths with weights). ALIS makes the photons work for all the wavelengths at once. They can be combined: REPTRAN chooses the wavelengths, ALIS shares the photons among them.
- **Uneven sampling.** The starting wavelength is uniform by default. `wavelength_proba` can change this, and in the default mode each wavelength is normalised by its own photon count, so that is safe there. For ALIS it could not be confirmed from the code that the weights are corrected, so it should be tested before use.

7. What has been tried in this project

It runs. ALIS compiles and runs on this project's 3D voxel cloud scene (`Smartg(back=True, opt3d=True, alis=True, alt_pp=True)`), with plain wavelengths and with REPTRAN band objects (the MODIS Terra channels b03, b04, b01, one internal wavelength each).

Comparison with the default mode. Same cropped BOMEX cloud, same view, 32×32 pixels, 20 000 photons per pixel, the three MODIS channels in one run. `Alis(n_low=-1)`, i.e. the scattering is computed at every wavelength. Two runs of each with different seeds; the noise of one run is taken from the difference between the two (rms / √2).

	Blue (b03)	Green (b04)	Red (b01)
Noise, default mode	0.0348	0.0319	0.0243
Noise, ALIS	0.0314	0.0170	0.0182
Noise ratio ALIS / default	0.90	0.53	0.75
Mean reflectance, default	0.1334	0.1019	0.0903
Mean reflectance, ALIS	0.1360	0.1030	0.0907

The mean run time was the same: 67 s in the default mode against 68 s with ALIS. At equal time the noise-based efficiency (1 / noise^2 per second) was therefore about 1.2, 3.4 and 1.7 times higher with ALIS, for blue, green and red.

Not established. This is a single geometry and one pair of runs per mode, so the ratios carry a statistical uncertainty that was not quantified, and the uneven gain across the channels is unexplained. The mean reflectances agree to within about 2%, but the blue difference (0.1360 against 0.1334) is roughly twice its statistical uncertainty, so a small bias cannot be excluded. The ALIS assumption (scattering nearly the same at all wavelengths) does not hold between 465 and 646 nm. A reliable verdict needs more photons, a high-statistics default-mode reference, and a pixel-by-pixel comparison. The playground does not use ALIS.

References and sources

• C. Emde, R. Buras, B. Mayer, *ALIS: An efficient method to compute high spectral resolution polarized solar radiances using the Monte Carlo approach*, Journal of Quantitative Spectroscopy and Radiative Transfer (2011). SMART-G's documentation cites it as “Emde et al. 2010”; the exact citation was written from memory and should be checked.
• SMART-G 2.0.0b1 sources read for this note: `smartg.py` (class Alis, option alis, alt_pp), `src/device.cu` (photon launch and counters), `bandset.py` (spectral_grids), `histories.py`.